

Listing 16-5: The `handleInputs()` method of the `ScenePlay` class

The `handleInputs()` method ❶ is responsible for click events. If the state machine is in the playing state, the user cannot click the buttons, so we don't bother checking for events ❷. If the user presses New Game ❸, we call `reset()` to reinitialize variables and change the state machine to the playing state. If the user presses Go to High Scores ❹, we navigate to the High Scores scene using the inherited `self.goToScene()` method. If the user toggles the Background Music checkbox ❺, we call its `getValue()` method to retrieve its new setting; the `reset()` method uses this setting to decide if background music should be played. If the user presses Quit ❻, we call the inherited `self.quit()` method from the base class. [Listing 16-6](#) shows the code for the actual game play.

```
❶ def update(self):
    if self.playingState != STATE_PLAYING:
        return # only update when playing

    # Move the Player to the mouse position, get back its rect
    ❷ mouseX, mouseY = pygame.mouse.get_pos()
    playerRect = self.oPlayer.update(mouseX, mouseY)

    # Tell the GoodieMgr to move all Goodies
    # Returns the number of Goodies that the Player contacted
    ❸ nGoodiesHit = self.oGoodieMgr.update(playerRect)
    if nGoodiesHit > 0:
        self.dingSound.play()
        self.score = self.score + (nGoodiesHit * POINTS_FOR_GOODIE)

    # Tell the BaddieMgr to move all the Baddies
    # Returns the number of Baddies that fell off the bottom
    ❹ nBaddiesEvaded = self.oBaddieMgr.update()
    self.score = self.score + (nBaddiesEvaded * POINTS_FOR_BADDIE_EVADED)
    self.scoreText.setValue(self.score)

    # Check if the Player has hit any Baddie
    ❺ if self.oBaddieMgr.hasPlayerHitBaddie(playerRect):
        pygame.mouse.set_visible(True)
        pygame.mixer.music.stop()

        self.gameOverSound.play()
```